

Department of Artificial Intelligence and Machine Learning

Date: 13.01.2026	Improvement Quiz and Test	Max. Marks: 50+10
Semester: V	UG	Duration: 1 $\frac{1}{2}$ Hrs + $\frac{1}{2}$ Hr
Course Title: Artificial Neural Networks and Deep Learning		Course Code: AI253IA

Part A

Sl No	Questions	M	BT	CO
1.	Apply the concept of exploration vs exploitation to a recommendation system.	2	L3	3
2.	Differentiate between the Naïve algorithm and Greedy algorithm in multi-armed bandits.	2	L4	2
3.	Write the mathematical expression for the confidence bonus used in the Upper Confidence Bound (UCB) algorithm and explain its purpose.	2	L3	2
4.	Define the Q-function in a Markov Decision Process and state what information it provides to the agent.	1	L2	1
5.	Write the min-max objective function of a Generative Adversarial Network and state the role of the generator in this formulation.	2	L3	2
6.	Identify the main components of a Neural Turing Machine and state the function of its external memory.	1	L1	1

Part B

Sl No	Questions	M	BT	CO
1a.	Describe stateless reinforcement learning algorithms with emphasis on the Multi-Armed Bandit problem.	5	L2	1
1b.	Analyze how Upper Bounding Methods balance exploration and exploitation.	5	L4	2
2a.	Analyze the differences between modeling states and state-action pairs in reinforcement learning.	5	L4	2
2b.	Evaluate the effectiveness of SARSA in stochastic environments compared to off-policy methods.	5	L3	3
3a.	Discuss the AlphaGo and AlphaZero systems as case studies in deep reinforcement learning. Your answer should include: Role of deep neural networks, Use of self-play, Impact of removing human knowledge	5	L2	3
3b.	Analyze the role of attention mechanisms in machine translation systems.	5	L4	2
4a.	Critically evaluate the limitations of neural networks, particularly in safety-critical applications such as self-driving cars.	5	L2	5
4b.	Explain the concept of Conditional GANs (cGANs). How do they extend the basic GAN architecture to allow for more controlled generation of images based on specific conditions (e.g., class labels, input images)?	5	L2	2
5a.	A coach must choose between Player A and Player B. Future performance is discounted. i) Goals next season are worth 80% of current goals. ii) Goals two seasons later are worth 64% of current goals. Player A scores 10 goals in each of the next three seasons. Player B scores 25 goals this season and 5 goals next season. - Determine the discount factor (γ). - Calculate the discounted return for each player. Which player should the coach select? Justify your answer.	5	L3	3
5b.	Explain the architecture and training process of Generative Adversarial Networks (GANs).	5	L2	2

CIE 3 - Quiz Solutions

Date: 13th January 2026

1. Apply the concept of exploration vs exploitation to a recommendation system.

In the context of a recommendation system (e.g., Netflix, Spotify):

- **Exploitation:** The system recommends content that it *knows* the user will like based on their past history (e.g., recommending another sci-fi movie because the user watched three sci-fi movies last week). This maximizes immediate user satisfaction and engagement.
- **Exploration:** The system recommends content that the user has *not* interacted with before or belongs to genres the user hasn't tried (e.g., recommending a documentary to a user who usually watches comedies).
 - **Purpose:** This gathers new data about the user's preferences. Without exploration, the system traps the user in a "filter bubble" and fails to discover latent interests, potentially leading to long-term boredom.

2. Differentiate between the naive algorithm and greedy algorithm in multi-armed bandits.

Feature	Naive Algorithm (Random)	Greedy Algorithm
Strategy	Selects an arm (action) completely at random at every step.	Selects the arm with the highest estimated value (average reward) observed so far at every step.
Learning	Does not use past data to improve decision-making.	Uses past data to exploit the current best option.
Flaw	Fails to accumulate high rewards because it picks bad arms as often as good ones.	Suboptimal Convergence: It never explores. If the "best" arm so far is actually mediocre (due to bad luck in early samples), the greedy algorithm gets stuck picking it forever and never finds the true optimal arm.

3. Write the mathematical expression for the confidence bonus used in the Upper Confidence Bound (UCB) algorithm and explain its purpose.

Expression: The Upper Confidence Bound for an action a at time t is calculated as $Q_t(a) + U_t(a)$. The **confidence bonus** term is:

$$\text{Bonus} = \sqrt{\frac{2 \ln t}{N_a(t)}}$$

Where:

- t = Total number of trials so far.
- $N_a(t)$ = Number of times action a has been selected.

Purpose: This term represents the **uncertainty** regarding the action's value.

- If an action is rarely chosen (N_a is small), the bonus is **large**, inflating the action's value to encourage the agent to **explore** it.
- As an action is chosen more often (N_a increases), the bonus shrinks, and the algorithm relies more on the actual average reward (exploitation).

4. Define the Q-function in a Markov Decision Process (MDP) and state what information it provides to the agent.

Definition: The Q-function (Action-Value Function), denoted as $Q^\pi(s, a)$, represents the **expected return** (cumulative discounted reward) starting from state s , taking action a , and following policy π thereafter.

$$Q^\pi(s, a) = \mathbb{E}_\pi[R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots | S_t = s, A_t = a]$$

Information Provided: It tells the agent **how good it is to take a specific action in a specific state**. Unlike the Value function $V(s)$ which just says how good a state is, $Q(s, a)$ allows the agent to construct a policy by simply choosing the action with the highest Q-value in any given state.

5. Write the Min-Max objective function of a Generative Adversarial Network (GAN) and state the role of the generator in this formulation.

Objective Function:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

Role of the Generator (G): The Generator aims to **minimize** this function (specifically the second term).

- It tries to generate data $G(z)$ such that the Discriminator D predicts it is real (output close to 1).
- Mathematically, it wants to minimize $\log(1 - D(G(z)))$, which is equivalent to maximizing the probability that the Discriminator is fooled.

6. Identify the main components of a Neural Turing Machine (NTM) and state the function of its external memory.

Main Components:

1. **Controller:** A neural network (often an RNN or LSTM) that receives inputs and emits outputs.
2. **Read/Write Heads:** Mechanisms that perform operations on the memory matrix.
3. **External Memory Matrix:** A large matrix of memory slots where data can be stored.

Function of External Memory: The external memory allows the neural network to store and retrieve information over long time scales without interference. Unlike the internal state of an RNN (which is tied to computation and degrades over time), the NTM's external memory acts like RAM in a computer, enabling the model to perform complex algorithmic tasks (like copying or sorting sequences) that require maintaining precise data outside the neural network's weights.

Part B: Detailed Concepts (5 Marks)

1.a. Describe stateless reinforcement learning algorithms with emphasis on the multi-armed bandit problem.

Stateless Reinforcement Learning: In standard Reinforcement Learning (RL), an agent moves between different states ($S_t \rightarrow S_{t+1}$) based on its actions. In **Stateless RL**, there is no concept of state transitions. The environment is either static or resets after every action. The agent receives an input (or just an opportunity to act), chooses an action, receives a reward, and the episode ends immediately. The goal is simply to figure out which action yields the highest average reward without worrying about long-term future states.

The Multi-Armed Bandit (MAB) Problem: The classic example of stateless RL is the Multi-Armed Bandit problem.

- **The Scenario:** An agent is faced with a row of k slot machines ("one-armed bandits").
- **The Unknown:** Each machine provides a reward drawn from a specific, unknown probability distribution. Some machines pay out often; others rarely.
- **The Goal:** The agent has a limited number of pulls (time steps). It must determine which machine has the highest expected reward and pull it as many times as possible to maximize the total cumulative reward.
- **The Challenge:** Since the probability distributions are unknown, the agent must *explore* (pull different levers to gather data) and *exploit* (pull the lever currently believed to be the best).

Key Characteristics of MAB Algorithms:

1. **Value Estimation:** They maintain an estimated value $Q(a)$ for each action, usually the average of all rewards received from that action so far.
2. **Selection Rule:** They use a specific rule (like ϵ -greedy or UCB) to decide which arm to pull based on the current estimates.
3. **Regret Minimization:** The performance is measured by "Regret"—the difference between the reward the agent actually got and the reward it *could* have gotten if it had known the optimal arm from the start.

1.b. Analyze how upper bounding methods balance exploration and exploitation.

Upper Bounding methods, specifically the **Upper Confidence Bound (UCB)** algorithm, balance exploration and exploitation by following the principle of "**Optimism in the Face of Uncertainty.**"

Instead of just looking at the *average* reward of an arm (which would be pure exploitation), UCB looks at the *potential* of an arm. It calculates a score based on two components:

$$A_t = \arg \max_a \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right]$$

1. Exploitation (The $Q_t(a)$ Term):

- This term represents the estimated value of the action based on past experience (the average reward).
- If an arm has historically provided high rewards, this term will be large.
- Focusing on this term drives the agent to select the "best known" arm, maximizing immediate returns.

2. Exploration (The Confidence/Bonus Term):

- This term $\left(\sqrt{\frac{\ln t}{N_t(a)}} \right)$ represents the uncertainty or variance of the estimate.
- $N_t(a)$ is the number of times action a has been chosen.
- **How it forces exploration:** If an arm has been chosen very few times ($N_t(a)$ is small), the denominator is small, making the total bonus term **very large**. Even if the average reward $Q_t(a)$ is low, the large bonus can push the total score above the "current best" arm, forcing the agent to try it.
- **Concept:** "I haven't tried this arm much, so for all I know, it *could* be amazing. I will give it the benefit of the doubt."

Analysis of the Balance Dynamics:

- **Early Phase:** When t is small, all arms have been tried few times. The uncertainty is high for everyone, leading to high exploration.
- **Correction:** When the agent explores a specific arm a , $N_t(a)$ increases. This increases the denominator, shrinking the bonus term. The uncertainty decreases.
- **Convergence:** As time goes on, the exploration bonus for frequently picked arms fades away. The algorithm naturally shifts from exploration to exploitation, eventually settling on the arm with the highest true Q -value as the uncertainty reaches zero.

2.a. Analyze the differences between modeling states and state-action pairs in reinforcement learning.

In Reinforcement Learning, we can track the value of being in a state (V) or the value of taking an action in a state (Q). This distinction fundamentally changes how agents select actions.

1. Modeling States ($V(s)$ - State-Value Function):

- **Definition:** $V(s)$ estimates the expected return starting from state s and following a specific policy thereafter. It tells you "how good is it to be here?"
- **Dependency:** It is a function of the state only.
- **Limitation for Control:** Knowing $V(s)$ alone is not enough to decide what to do next. To pick the best action, the agent needs a model of the environment's dynamics (transition probabilities $P(s'|s, a)$).
 - *Equation:* To act, you must calculate: $\arg \max_a \sum_{s'} P(s'|s, a)[R + \gamma V(s')]$.
 - *Constraint:* This requires "lookahead" capabilities or a known model.

2. Modeling State-Action Pairs ($Q(s, a)$ - Action-Value Function):

- **Definition:** $Q(s, a)$ estimates the expected return starting from state s , taking action a , and following a specific policy thereafter. It tells you "how good is it to do *this* specific thing right *here*?"
- **Dependency:** It is a function of both state and action.
- **Advantage for Control (Model-Free):** The primary advantage is that it allows for **Model-Free Control**.
 - If an agent knows the Q -values, it does not need to know the transition probabilities (P) or the reward function (R) to act.
 - *Selection:* The agent simply looks at all available actions in the current state and picks the one with the highest Q -value: $\arg \max_a Q(s, a)$.
 - The "lookahead" (the consequences of the action) is essentially cached directly inside the Q -value.

2.b. Evaluate the effectiveness of SARSA in stochastic environments compared to policy methods.

SARSA (State-Action-Reward-State-Action): SARSA is an **On-Policy** TD learning algorithm. Its update rule depends on the tuple (S, A, R, S', A') , where A' is the action *actually taken* in the next state according to the current policy (including exploration steps).

$$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma Q(S', A') - Q(S, A)]$$

Effectiveness in Stochastic Environments (The Safety Factor): In environments where actions have stochastic outcomes or where the agent must explore (e.g., using ϵ -greedy), SARSA evaluates the policy *as it is being executed*, not the theoretical optimal policy.

Comparison with Off-Policy Methods (like Q-Learning):

1. Risk Sensitivity:

- **Q-Learning (Off-Policy):** Assumes the agent will always take the optimal action in the next step ($\max_a Q(s', a)$). In a stochastic environment (like the "Cliff Walking" problem), Q-learning might learn a path that walks right along the edge of a cliff because that is the shortest optimal path. However, if the agent is still exploring ($\epsilon > 0$), it has a high chance of stepping off the cliff due to randomness.
- **SARSA (On-Policy):** Learns the value of the *actual* behavior, which includes the random ϵ exploration steps. SARSA "realizes" that walking near the cliff is dangerous because there is a non-zero probability (due to exploration) of falling off. Therefore, SARSA tends to learn a **safer, more conservative path** deeper inside the grid, avoiding the edge.

2. Online Performance:

- In scenarios where training performance matters (accumulated reward *during* learning), SARSA often outperforms Q-learning in stochastic environments because it avoids high-penalty states that Q-learning risks encountering.

3. Convergence:

- Q-learning converges to the *optimal* policy directly.
- SARSA converges to a *near-optimal* policy that accounts for exploration. It only converges to the true optimal policy if the exploration rate (ϵ) slowly decays to zero over time.

3.a. Discuss the AlphaGo and AlphaZero systems as case studies in Deep Reinforcement Learning.

1. Role of Deep Neural Networks: Both systems utilize Deep Convolutional Neural Networks (CNNs) to process the game board (a grid of 19x19) similar to how CNNs process images.

- **AlphaGo:** Used two separate networks.
 - **Policy Network:** Predicted the next move (probability distribution over legal moves).
 - **Value Network:** Predicted the winner of the game from the current position (a scalar value).
- **AlphaZero:** Streamlined this into a **single neural network** with two "heads" (one for policy, one for value). This improved efficiency and allowed the policy and value estimates to share feature representations.

2. Use of Self-Play: Self-play is the core training mechanism that allows these systems to exceed human performance.

- Instead of learning from a fixed dataset of static games, the agent plays millions of games against **versions of itself**.
- **Curriculum Learning:** As the agent improves, its opponent (itself) also improves. This creates an auto-curriculum where the difficulty perfectly matches the agent's current skill level, preventing the agent from plateauing.
- The data generated from these self-play games (states, moves taken, and final winners) is used to continuously retrain the neural networks.

3. Impact of Removing Human Knowledge (AlphaZero):

- **AlphaGo (Original):** Relied heavily on **Supervised Learning** from thousands of human expert games to initialize its policy network. It essentially started by mimicking humans.
- **AlphaZero:** Removed all human data. It started with random weights and no knowledge of the game beyond the basic rules.
 - **Result:** It not only surpassed AlphaGo but discovered "alien" strategies and opening moves that humans had discarded or never discovered in 2,500 years of Go history.
 - **Significance:** This proved that human data can actually be a bottleneck (bias). Pure RL via self-play can find optimal strategies that are counter-intuitive to human experts.

3.b. Analyze the role of attention mechanisms in Machine Translation systems.

The Bottleneck Problem in Encoder-Decoder Models: Traditional Recurrent Neural Networks (RNNs) used for translation encoded the entire input sentence (e.g., "The cat sat on the mat") into a **single fixed-length vector** (the context vector).

- **Issue:** Compressing a long sentence (e.g., 50 words) into one vector causes information loss, specifically regarding the earlier parts of the sentence (Vanishing Gradient problem).

The decoder essentially "forgets" the beginning of the sentence by the time it starts generating the translation.

The Role of Attention: Attention mechanisms solve this by allowing the decoder to **"look back"** at the entire input sequence at every single decoding step, rather than relying on one static vector.

1. **Dynamic Context:** At each step of generating the translation, the Attention mechanism creates a unique context vector.
2. **Alignment Scores (Weights):** It calculates alignment scores that determine which words in the input sentence are most relevant *right now*.
 - *Example:* When translating "La maison" to "The house," when the decoder is about to generate "house," the attention mechanism assigns a high weight to the input vector for "maison" and low weights to other words.
3. **Visualization:** This creates an alignment matrix (heatmap) showing which source words correspond to which target words, effectively solving the alignment problem implicitly.

Impact:

- **Long Dependencies:** It drastically improves performance on long sentences because the model doesn't need to "remember" everything in one vector; it can just "look up" the information when needed.
- **Parallelization:** The success of Attention led to the **Transformer** architecture (which is purely attention-based), allowing for massive parallelization and the training of modern Large Language Models (LLMs) like BERT and GPT.

4.a. Critically evaluate the limitations of neural networks, particularly in safety-critical applications such as self-driving cars.

While Neural Networks (NNs) achieve state-of-the-art performance in perception tasks, their application in safety-critical systems like autonomous vehicles reveals significant limitations:

1. Lack of Interpretability (The "Black Box" Problem):

- Deep learning models map inputs to outputs through millions of parameters. When a car makes a mistake (e.g., swerving unexpectedly), it is often impossible to trace *why* the network made that decision.
- **Safety Impact:** If engineers cannot understand the "logic" of the failure, they cannot guarantee a fix. This makes certification and auditing extremely difficult compared to rule-based software.

2. Adversarial Attacks:

- NNs are highly susceptible to adversarial examples—inputs with small, often imperceptible perturbations designed to fool the model.
- **Safety Impact:** A malicious actor could place a small sticker on a "Stop" sign that looks like graffiti to a human but causes the car's vision system to classify it as a "Speed Limit 45" sign, leading to potentially fatal accidents.

3. Generalization to Out-of-Distribution (OOD) Data:

- NNs assume the real world follows the same distribution as their training data. However, the real world is chaotic (rare weather, odd construction zones, debris).
- **Safety Impact:** When a self-driving car encounters a "corner case" it hasn't seen before (e.g., a person in a dinosaur costume crossing the street), the model's behavior is unpredictable. Unlike humans, who use common sense, the NN might confidentially make a catastrophic error.

4. Overconfidence:

- The Softmax output of a classification network represents probabilities, but these are not reliable measures of *confidence*. A model can predict a wrong class with 99.9% probability.
- **Safety Impact:** The system might fail to hand over control to the human driver because it "thinks" it knows what it is doing, right up until the moment of impact.

4.b. Explain the concept of Conditional GANs (cGANs). How do they extend the basic GAN architecture?

Concept: A standard Generative Adversarial Network (GAN) generates images from random noise (z). You have no control over what specific image is produced (e.g., if trained on digits, it might produce a '1' or a '9' randomly). **Conditional GANs (cGANs)** introduce a control mechanism. They condition the model on additional information (y), such as class labels or data from other modalities, allowing for the generation of specific types of images.

Architecture Extension:

1. Conditioning the Generator (G):

- **Input:** Instead of just noise z , the input is the combined vector (z, y) .
- **Function:** $G(z, y) \rightarrow x_{fake}$. The generator learns to map noise to the data space given the condition y .
- *Example:* If y is the label "Cat", the generator must produce an image of a cat, not a dog.

2. Conditioning the Discriminator (D):

- **Input:** The discriminator receives both the image x (real or fake) and the condition y .
- **Function:** $D(x, y) \rightarrow [0, 1]$.

- **Task:** The discriminator must check two things:
 1. Is the image x real?
 2. Does the image x match the label y ?
- *Mechanism:* If the Generator produces a realistic-looking "Dog" when the label y was "Cat", the Discriminator will penalize it, forcing the Generator to respect the condition.

Applications:

- **Text-to-Image Synthesis:** Condition on a text description ("A red bird with a yellow beak").
- **Image-to-Image Translation (Pix2Pix):** Condition on an input image (e.g., converting a sketch to a photo, or a black-and-white photo to color).

5.a. A coach must choose between Player A and Player B based on Discounted Return.

Given Data:

- **Discounting Logic:**
 - Next season (1 year away) = 80% worth $\rightarrow$ Discount factor $\gamma = 0.8$.
 - Two seasons later (2 years away) = 64% worth $\rightarrow \gamma^2 = (0.8)^2 = 0.64$.
 - **Confirmed:** $\gamma = 0.8$.
- **Player A Performance:** 10 goals in each of the next 3 seasons (Assume $t = 0, 1, 2$).
- **Player B Performance:** 25 goals this season ($t = 0$) and 5 goals next season ($t = 1$).

Calculations:

1. Player A Return (G_A):

$$G_A = R_0 + \gamma R_1 + \gamma^2 R_2$$

$$G_A = 10 + (0.8 \times 10) + (0.8^2 \times 10)$$

$$G_A = 10 + 8 + 6.4$$

$$G_A = 24.4$$

2. Player B Return (G_B):

$$G_B = R_0 + \gamma R_1 + \gamma^2 R_2$$

$$G_B = 25 + (0.8 \times 5) + (0.64 \times 0)$$

$$G_B = 25 + 4 + 0$$

$$G_B = 29.0$$

Conclusion: The coach should select **Player B**. **Justification:** Even though Player A is consistent over a longer period, Player B provides a significantly higher immediate reward (25 goals now). Because the coach discounts future performance (valuing immediate success more), Player B's higher initial output results in a higher Total Discounted Return ($29.0 > 24.4$).

5.b. Explain the architecture and training process of Generative Adversarial Networks (GANs).

Architecture: A GAN consists of two distinct neural networks contesting with each other in a zero-sum game framework:

1. The Generator (G):

- **Goal:** To generate "fake" data (e.g., images) that look indistinguishable from real data.
- **Input:** A random noise vector (z) from a latent space.
- **Output:** A generated sample $x' = G(z)$.

2. The Discriminator (D):

- **Goal:** To distinguish between real data from the dataset and fake data produced by the Generator.
- **Input:** A data sample x .
- **Output:** A probability $D(x)$ indicating whether x is real (1) or fake (0).

Training Process (Minimax Game): The training involves alternating steps where the networks compete:

Step 1: Train the Discriminator (D)

- Freeze the Generator.
- Feed D a batch of **Real Data** (Label = 1).
- Feed D a batch of **Fake Data** generated by G (Label = 0).
- **Update D :** Adjust weights to maximize classification accuracy (minimize binary cross-entropy loss). D learns to spot the flaws in G 's creations.

Step 2: Train the Generator (G)

- Freeze the Discriminator.

- Generate a batch of fake images.
- Feed these images to D .